

字符串方法

4.1.2 字符串方法

- 去空白字符 : `strip([chars])` `lstrip([chars])` `rstrip([chars])`
- 字符串类型判断 : `isalnum()` `isalpha()` `isdecimal()` `isdigit()` `isidentifier()` `islower()` `isupper()` `isnumeric()` `isprintable()` `isspace()` `istitle()`
- 大小写转换: `lower()` `upper()` `swapcase()` `capitalize()` `title()` `casefold()`
- 测试和查找: `startswith(prefix[,start[,end]])` `endswith(suffix[,start[,end]])` `count(sub[,start[,end]])` `index(sub[,start[,end]])` `rindex(sub[,start[,end]])` `find(sub[,start[,end]])` `rfind(sub[,start[,end]])`
- 替换: `replace(old,new[,count])`
- 拆分和组合: `split(sep=None,maxsplit=-1)` `rsplit(sep=None, maxsplit=-1)` `partition(sep)` `rpartition(sep)` `splitlines([keepends])` `join(iterable)`
- 填充和对齐 : `zfill(width)` `center(width[,fillchar])` `ljust(width[,fillchar])` `rjust(width[,fillchar])` `expandtabs([tabsize])`
- 翻译和转换 : `maketrans` `translate`

去空白字符

- strip([chars]) lstrip([chars])rstrip([chars])
- 将字符串的两端、左端或者右端的空格去掉后返回一个新的字符串。如果指定了chars, 则为去掉字符串两端、右端或左端开始处chars中的那些字符。

```
>>> s=" abc "  
>>> s1=s.strip( )  
>>> s1  
"abc"  
>>> s2 = s.lstrip( )  
>>> s2  
"abc "  
>>> s3 = s.rstrip( )  
>>> s3  
" abc"
```

```
>>> "aaaassddf".strip("a")  
"ssddf"  
>>> "aaaassddf".strip("asf")  
"dd"  
>>> "aaaassddfaaa".rstrip("a")  
'aaaassddf'  
>>> "aaaassddfaaa".lstrip("a")  
'ssddfaaa'
```

字符串类型判断

- `isdigit()`: 字符串是否都为数字字符
- `isalpha()` : 字符串是否都为字母(注这里指的是Unicode范围)
- `isalnum()` : 字符串为字母或者numeric
- `isidentifier()`: 是否可以作为Python标识符
- `islower()` `isupper()` : 是否为小写或者大写字母
- `isprintable()` : 是否为可打印字符
- `isspace()`: 是否为空格 (包括制表符`\t`、回车`\n`等)
- `istitle()`: 每个单词的首字母是否为大写

字符串类型判断:例子

- 注意数字、字母字符的判断都是Unicode字符集范围

```
>>> 'abcd'.isalpha()
True
>>> '1234'.isalpha()
False
>>> '一千三百'.isalpha()
True

>>> '1234abcDE'.isalnum()
True
```

```
>>> '中国一千三百'.isidentifier()
True
>>> '1234abcDE'.isidentifier()
False

>>> '1234abcDE'.islower()
False
>>> 'abcde'.islower()
True
>>> 'ABCDE'.isupper()
True

>>> '\t\n'.isspace()
True

>>> 'How Are You?'.istitle()
True
```

大小写转换

- `lower()` 得到小写格式字符串
- `upper()` 得到大写格式字符串
- `capitalize()` 得到首字母大写格式
- `title()` 得到每个单词首字母大写格式
- `swapcase()` 得到大小写互换格式

```
>>> s="What is YoUr NaMe?"
>>> s2=s.lower()
>>> s2
'what is your name?'
>>> s
'What is YoUr NaMe?'
>>> s.upper()
'WHAT IS YOUR NAME?'
>>> s2.capitalize()
'What is your name?'
>>> s2.title()
'What Is Your Name?'
>>> s.swapcase()
'wHAT IS yOuR nAmE?'
```

4.1.3 字符串常量: string模块

```
>>> import string
>>> string.digits          #数字字符
'0123456789'
>>> string.punctuation    #标点符号
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
>>> string.ascii_letters   #英文字母
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz'
>>> string.printable       #可打印字符
'0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#$%&\'()*+,-./:;<=>?@[\]^_`{|}~ \t\n\r\x0b\x0c'
>>> string.ascii_lowercase #小写字母
'abcdefghijklmnopqrstuvwxyz'
>>> string.ascii_uppercase #大写字母
'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

练习1：删除字符串中的非字母和非数字的符号

```
import string
s = input("请任意输入：")

charSet = string.ascii_letters \
        + string.digits

t = ""
for x in s:
    if x in charSet:
        t = t + x
print(t)
```

```
>>>
请任意输入： asdf330* () 349/;asd&) (232Uj
asdf330349asd232Uj
>>>
```

练习2:自己实现大小写转换

```
def swap_case(ch):  
    if 'a' <= ch <= 'z':  
        return chr(ord(ch) - ord('a') + ord('A'))  
    if 'A' <= ch <= 'Z':  
        return chr(ord(ch) - ord('A') + ord('a'))  
    return ch
```

```
import string  
def swap_case_2(ch):  
    if ch in string.ascii_lowercase:  
        return chr(ord(ch) - ord('a') + ord('A'))  
    if ch in string.ascii_uppercase:  
        return chr(ord(ch) - ord('A') + ord('a'))  
    return ch
```

练习3:统计文章的单词个数。约定单词由英文字母组成，其他字符只是用来分隔单词。

```
import string
def count_words(sentence='If winter comes, can spring be far behind?'):
    words = 0
    inword = False
    for c in sentence:
        if inword:
            if c not in string.ascii_letters:
                inword = False
        else:
            if c in string.ascii_letters:
                inword = True
                words += 1
    return words
```

请输入多个英文单词，可以用任何非英文字母分割单词
If winter comes, can spring be far behind?
8

```
def test_count_words():
    s = input('请输入多个英文单词，可以用任何非  
英文字母分割单词\n')
    words = count_words(s)
    print('英文单词个数:%d' % words)
```

查找: startswith endswith

startswith(prefix[,start[,end]])

endswith(suffix[,start[,end]])

- 是否以prefix开头或者suffix结尾
- prefix和suffix可以是字符串; 也可以是多个字符串 (tuple) ,表示其中任意一个字符串开头或者结尾

```
>>> import os
>>> [filename for filename in os.listdir(r'c:\windows' ) if
filename.endswith(('.exe','.bat'))]
['bfsvc.exe', 'evim.bat', 'explorer.exe', 'fveupdate.exe', 'gview.bat',
'gvim.bat', 'gvimdiff.bat', 'HelpPane.exe', 'hh.exe', 'notepad.exe', 'py.exe',
'pyw.exe', 'regedit.exe', 'splwow64.exe', 'twunk_16.exe', 'twunk_32.exe',
'view.bat', 'vim.bat', 'vimdiff.bat', 'vimtutor.bat', 'winhlp32.exe',
'write.exe']
```

成员判断和查找

- 有序序列都支持in,count,index, str也不例外
 - `sub in S` `sub not in S`: 判断子串sub (不仅仅是单个字符) 是否在S中出现。
 - `index(sub[,start[,end]])` `rindex(sub[,start[,end]])`: 返回子串sub在指定范围内首次或者最后一次出现的位置(下标), 如果不存在则抛异常ValueError
 - `count(sub[,start[,end]])`: 返回子串sub出现的次数
- `find(sub[,start[,end]])` `rfind(sub[,start[,end]])`: 和index类似, 返回子串sub在指定范围内首次或者最后一次出现的位置, 如果不存在则返回-1

成员判断和查找例子

```
>>> s="apple,peach,banana,peach,pear"
>>> 'c' in s
True
>>> 'pear' in s
True
>>> 't' not in s
True
>>>

>>> s.index('peach')
6
>>> s.index('ppp')
Traceback (most recent call last):
  File "<pyshell#11>", line 1, in <module>
    s.index('ppp')
ValueError: substring not found

>>> s.rindex('peach')
19
```

```
>>> s.count('p')
5
>>> s.count('pp')
1
>>> s.count('ppp')
0

>>> s.find("peach")
6
>>> s.find("peach",7)
19

>>> s.find("peach",7,20)
-1
>>> s.rfind('p')
25
>>> s.rfind('ppp')
-1
```

字符串替换

replace(old,new[,count]) : 返回**新的**字符串，默认原始字符串中的**所有**子串old被替换为new。如果count给出，则仅替换前面的count个子串

```
>>> s="apple,peach,banana,peach,pear"
>>> s.replace('peach','PEACH')
'apple,PEACH,banana,PEACH,pear'
>>> s.replace('peach','PEACH',1)
'apple,PEACH,banana,peach,pear'

>>> s.replace('lemon','LEMON')
'apple,peach,banana,peach,pear'
>>>
```

分割和组合:

- `split(sep=None,maxsplit=-1)` `rsplit(sep=None, maxsplit=-1)`
- `partition(sep)` `rpartition(sep)`
- `splitlines([keepends])`
- `join(iterable)`

分割和组合: split

`split(sep=None,maxsplit=-1)` `rsplit(sep=None, maxsplit=-1)`

- 以指定字符串sep为分隔符分割多个字符串，返回包含分割结果的列表。sep一般为单个字符，但也可以是多个字符。如果字符串中sep连续出现，或出现在字符串最前面和最后面，这些sep之间会分割出空字符串。
- 如果maxsplit给出且不为-1，则只进行前面maxsplit次分割，即列表元素最多maxsplit+1。如果为-1，则不限次数
- 如果sep=None，则所有空白符号（包括空格、换行符、制表符等等）都将被认为是分隔符，而且多个连续的空白符号当成一个空白分隔符。字符串最前面和最后面分割出来的空字符串会被移走

split例子

```
>>> s="  apple , peach , banana,,peach ,pear ,"
>>> s.split(',')
['  apple ', ' peach ', ' banana', '', 'peach ',
'pear ', '']

>>> [i.strip() for i in s.split(',')]
['apple', 'peach', 'banana', '', 'peach', 'pear',
'']

>>> [i.strip() for i in s.split(',') if i != '']
['apple', 'peach', 'banana', 'peach', 'pear']

>>> s.split(', ', 2)
['  apple ', ' peach ', ' banana,,peach ,pear ,']

>>> s.split('peach')
['  apple ', ' ', ' ', banana,,', ' ', pear ,']

>>>
```

```
>>> t= "2014-10-31".split("-")
>>> print(t)
['2014', '10', '31']
>>> print([int(c) for c in t])
[2014, 10, 31]

>>> s2 = '\nHello world \n\n My  name
is Mike  '
>>> s2.split()
['Hello', 'world', 'My', 'name', 'is',
'Mike']

>>> s2.split(' ')
['\nHello', 'world', '\n\n', 'My', '',
'name', 'is', 'Mike', '', '', '']
```

Review:1.9 Python快速入门

- 问题3：任意输入三个英文单词，按它们在字典中顺序输出。

```
s = input('x,y,z=')
x, y, z = s.split(',') #字符串那章学后就可以理解
if x > y:
    x, y = y, x
if x > z:
    x, z = z, x
if y > z:
    y, z = z, y
print(x, y, z)
```

```
>>> s = input(' x, y, z=')
x, y, z=boy, like, map
>>> s
'boy, like, map'
>>> x, y, z = s.split(',')
>>> x
'boy'
>>> y
'like'
>>> z
'map'
>>>
```

分割和组合: partition rpartition

- **partition(sep) rpartition(sep)**

- 以指定字符串sep为分隔符将原字符串分割为包括3部分的元组，即sep前的字符串、sep、sep后的字符串，如果指定的分隔符不在原字符串中，则返回原字符串和两个空字符串。

```
s="  apple , peach , banana,,peach ,pear ,"  
>>> s.partition(',')  
( '  apple ', ', ', ' peach , banana,,peach ,pear , ' )  
>>> s.rpartition(',')  
( '  apple , peach , banana,,peach ,pear ', ', ', ' ' )  
>>> s.rpartition('banana')  
( '  apple , peach , ', 'banana', ',,peach ,pear , ' )
```

分割和组合: splitlines

- `splitlines([keepends=False])`
 - 字符串根据换行类字符(包括'\n')分割成多个子字符串，返回分割后的列表。字符串最后为换行字符时不会导致额外的空字符串。
 - 空字符串调用本函数时返回空列表
 - `keepends=True`表示保留换行字符。

```
>>> s3 = '\nab c\n\nde fgkl\n'
>>> s3.splitlines()
['', 'ab c', '', 'de fgkl']
>>> s3.splitlines(keepends=True)
['\n', 'ab c\n', '\n', 'de fgkl\n']
>>> s3.split('\n')
['', 'ab c', '', 'de fgkl', '']
>>> s3.split()
['ab', 'c', 'de', 'fgkl']
>>> ''.split('\n')
['']
>>> ''.splitlines()
[]
```

分割和组合: join

- **join(iterable)** : 将iterable对象(列表,字符串等)中的多个元素(是字符串类型)连接在一起组合成新的字符串, 元素间插入相应的字符串
- 不推荐使用 + 连接字符串, 优先使用join()方法
 - 效率原因

".join(list2)结果是什么 ?

```
>>> list1=["apple", "peach",  
"banana","pear"]  
>>> ','.join(list1)  
'apple,peach,banana,pear'  
  
>>> list2 = list(range(5))  
>>> list2  
[0, 1, 2, 3, 4]  
>>> '+'.join(map(str,list2))  
'0+1+2+3+4'  
>>> '+'.join([str(i) for i in list2])  
'0+1+2+3+4'  
  
>>> s4 = 'notice'  
>>> ' '.join(s4.upper())  
'N O T I C E'
```

练习4：自己实现大小写转换+

```
def swap_case(ch):  
    if 'a' <= ch <= 'z':  
        return chr(ord(ch) - ord('a') + ord('A'))  
    if 'A' <= ch <= 'Z':  
        return chr(ord(ch) - ord('A') + ord('a'))  
    return ch
```

```
import string  
def swap_case_2(ch):  
    if ch in string.ascii_lowercase:  
        return chr(ord(ch) - ord('a') + ord('A'))  
    if ch in string.ascii_uppercase:  
        return chr(ord(ch) - ord('A') + ord('a'))  
    return ch
```

```
def test_swap_case():  
    s = input('请输入一个字符串:')  
    ss = []  
    for ch in s:  
        ss.append(swap_case(ch))  
    ss = ''.join(ss)  
    print(ss)
```

填充和对齐

`center(width[,fillchar])`

`ljust(width[,fillchar])`

`rjust(width[,fillchar])`

- 返回新字符串，原字符串居中或左对齐或右对齐，并使用指定字符(默认空格)填充，保证长度至少为width。

- `zfill(width)`为右对齐，0填充,等价于 `rjust(width,'0')`

```
>>> s='Hello world!'
>>> s.zfill(20)
'00000000Hello world!'
>>> s.rjust(20, '0')
'00000000Hello world!'
...|
```

- `expandtabs([tabsize])`: `\t`转换为空格，缺省为8个

```
>>> s='Hello world!'
>>> s.center(20)
'      Hello world!      '
>>> s.center(20, '=')
'====Hello world!===='
>>> s.ljust(20, '=')
'Hello world!===== '
>>> s.rjust(20, '=')
'=====Hello world!'

>>> '1\t2\t3'.expandtabs()
'1          2          3'
>>> '1\t2\t3'.expandtabs(2)
'1 2 3'
```

练习4:字符串应用例子：图形打印

- 打印三角形图案

```
lines = int(input('请输入n:'))  
  
for i in range(lines):  
    stars = '*' * (i+1)  
    print(stars)
```

```
for i in range(lines):  
    stars = ' '.join('*' * (i+1))  
    print(stars)
```

请输入n:5

```
*  
**  
***  
****  
*****
```

```
*  
* *  
* * *  
* * * *  
* * * * *
```

字符串应用例子：图形打印

- 打印三角形图案 $n = 5$ ：靠右对齐

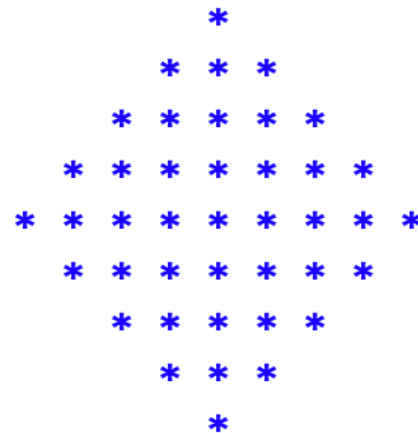
```
# n stars + n-1 space  
  
for i in range(lines):  
    stars = ' '.join('*' * (i+1))  
    line = stars.rjust(lines*2-1, ' ')  
    print(line)
```



请输入n:5

课堂练习题：用户输入一个正整数 n ，输出一个菱形

$n = 5$



字符串转换: maketrans translate(1)

str静态方法

maketrans(x,[y,[z]])生成映射表, 实际上就是一个dict

translate(table)按照映射表进行字符串转换

如果有两个以上参数, x和y必须长度一致, 每一个在x出现的字符转换为y对应位置的字符,

如果有第三个参数, 表示在z中出现的字符转换为None, 即删除

```
>>> s = 'Great hopes make great man.'
>>> tab =
str.maketrans('abcde', 'uvwxy')
>>> ss = s.translate(tab)
>>> ss
'Gryut hopys muky gryut mun.'
>>> tabb =
str.maketrans('uvwxy', 'abcde')
>>> ss.translate(tabb)
'Great hopes make great man.'
```

注意python3 这两个方法不再在

string模块中

字符串转换: maketrans translate(2)

```
>>> tab1 =  
str.maketrans('123456789', '*'*9)  
>>> s1.translate(tab2)  
'我的银行账号为***** , 密码为*****'
```

```
>>> s2 = 'mao@fudan.edu.cn'  
>>> tab3 = str.maketrans({'@': ' AT ',  
'.': ' DOT '})  
>>> s2.translate(tab3)  
'mao AT fudan DOT edu DOT cn'
```

```
>>> s1 = '我的银行账号为3141516, 密码为  
123456'  
>>> tab2 = str.maketrans('账号密码', '大  
学初中', '123456789')  
>>> tab1  
{30721: 23110, 23494: 22993, 36134:  
20061, 21495: 20843, 49: None, 50:  
None, 51: None, 52: None, 53: None, 54:  
None, 55: None, 56: None, 57: None}  
  
>>> s11 = s1.translate(tab1)  
>>> s11  
'我的银行大学为, 初中为'
```

4.1.1 字符串与数字之间的转换

- 数字→字符串：
 - `str(obj)`: `str(16)` `str(3.14)`
 - `bin/oct/hex`: 转变为二进制、八进制和十六进制字符串
 - 格式化 `%`、`format`方法等
- 字符串→数字
 - `int(x[,d])`: `d`进制的字符串转化为整数
 - `float(x)`: 转换为浮点数

`eval(source, globals=None, locals=None, /):`

`globals`和`locals`为表示某个namespace的dict，缺省为当前全局和局部scope。

`source`可以是表示Python表达式的字符串

eval示例

```
>>> eval("3+4")
7
>>> a = 3; b = 5
>>> eval("a+b")
8
>>> eval('print("a + b =",a+b)')
a + b = 8

>>> eval('c = a + b')
Traceback (most recent call last):
  File "<pyshell#859>", line 1, in
<module>
    eval('c = a + b')
    File "<string>", line 1
      c = a + b
      ^
SyntaxError: invalid syntax
```

Why?

只能是表达式，不能是语句

```
>>> import math
>>> eval('math.sqrt(3)')
1.7320508075688772

>>> eval('aa')
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in
<module>
    eval('aa')
    File "<string>", line 1, in <module>
NameError: name 'aa' is not defined
```

eval示例(补充, 但不作要求)

```
>>> cmd = "__import__('os').startfile(r'C:\\Windows\\notepad.exe')"  
>>> print(cmd)  
__import__('os').startfile(r'C:\\Windows\\notepad.exe')  
>>> eval(cmd)
```

- 上述代码不安全！！__import__为模块builtins对象 (__builtins__) 中的内置函数。eval时传递第二个参数时，第二个参数中字典的值优先

```
>>> import os  
>>>  
eval("os.startfile(r'C:\\Windows\\notepad.exe')")  
>>> safe_dict = {"__builtins__": {}, "os": None}  
>>>  
eval("os.startfile(r'C:\\Windows\\notepad.exe')",  
safe_dict)
```

```
>>> eval(cmd, {"__builtins__": {}})
```

eval仍然可能受到攻击！

```
>>> eval("' '*1000000*2*2*2*2*2*2*2*2*2*2")  
#CPU/Memory resource attack
```

eval示例(补充, 但不作要求)

- 建议采用ast模块
- Abstract Syntax Trees

```
import ast
ast.literal_eval( cmd )
```

```
import os
import traceback
import ast
def test_eval():
    cmd = "__import__('os').startfile(r'C:\\\\Windows\\\\notepad.exe') "
    input('eval %s...' % cmd)
    eval(cmd)
    cmd2 = "os.startfile(r'C:\\\\Windows\\\\notepad.exe') "
    input('eval %s...' % cmd2)
    eval(cmd2)
    try:
        safe_dict = {"__builtins__": {}, "os": None }
        input('eval with safe_dict...')
        eval(cmd2, safe_dict)
    except :
        traceback.print_exc()
    try:
        input('ast.literal_eval %s...' % cmd2)
        ast.literal_eval( cmd )
    except :
        traceback.print_exc()
```